Reference — DML / DDL / Queries

`schema: Departments(dept_id,dept_name,dept_head→emp,budget,city) · Employees(emp_id,fname,lname,salary,dept_id,manager_id)`

ADD / CHANGE / REMOVE

DML syntax & rules

```
INSERT INTO t (c1,c2) VALUES (v1,v2),(v3,v4);
INSERT INTO t (c1,c2) SELECT a,b FROM s WHERE ..; -- counts match
UPDATE t SET c1=v1,c2=v2 WHERE cond;
DELETE FROM t WHERE cond;
UPDATE t SET c=s.c FROM s WHERE t.k=s.k;   -- via other table
DELETE FROM t USING s WHERE t.k=s.k;
```

- **UPDATE:** WHERE picks rows first; SET right side uses **old** values. SET col has **no alias**, **no column list** after table.
- Omitted INSERT cols → DEFAULT, else NULL. DELETE has **no JOIN** (USING/subquery). Don't repeat target in UPDATE..FROM.
- **Traps: no WHERE = all rows · NOT IN w/ a NULL ⇒ nothing (use NOT EXISTS) · `=NULL` never true (use IS NULL) · scalar subquery = 1 row.**
- **ILIKE**=case-insensitive. `%`=any run, `_`=1 char; no % ⇒ whole string.

ROWS CHOSEN VIA ANOTHER TABLE

IN / EXISTS / anti-join

- **Has a match:** `WHERE key IN (SELECT key FROM other WHERE..)`.
- **Has none:** `NOT EXISTS (SELECT 1 FROM other o WHERE o.key=t.key)`. EXISTS needs the **correlation line**.
- **Insert only missing** (anti-join): `INSERT INTO t SELECT .. WHERE key NOT IN (SELECT key FROM t)`.
- Subquery in `SET` must return **1 row** (use SUM/MAX if several could match); a no-match correlated SET gives `qty + NULL = NULL` — guard with `IN`.

CHANGE STRUCTURE (ROWS KEPT)

ALTER TABLE (DDL)

```
ADD COLUMN c TYPE [NOT NULL] [DEFAULT v];
DROP COLUMN c [CASCADE];      RENAME COLUMN old TO new;
ALTER COLUMN c TYPE T [USING c::T];
ALTER COLUMN c SET DEFAULT v | DROP DEFAULT;
ALTER COLUMN c SET NOT NULL | DROP NOT NULL;
ADD CONSTRAINT n CHECK(..)|UNIQUE(c)|PRIMARY KEY(c)
  |FOREIGN KEY(c) REFERENCES p(k);   DROP CONSTRAINT n;
```

- Each is `ALTER TABLE t <action>;`. Several actions = comma-sep; RENAME can't combine. NOT NULL/DEFAULT use ALTER COLUMN. Change a constraint = DROP then ADD.
- **Fails: SET NOT NULL w/ NULLs · ADD CONSTRAINT w/ bad rows · TYPE won't fit (USING) · ADD COLUMN NOT NULL on filled table w/o DEFAULT.**
- DROP COLUMN blocked by a dependent **view** (CASCADE) **or an FK elsewhere** pointing at it; it also drops that column's own constraints.

PARENT DELETED/UPDATED

Foreign-key actions

Option	Child rows
`NO ACTION` (default)	reject if children exist
`CASCADE`	deleted too / get new key
`SET NULL`	key→NULL (not if in child PK)
`SET DEFAULT`	key→its default

- Change an action: DROP then ADD the FK (default name `table_col_fkey`). **Circular FK**: one col nullable + add FK later via ALTER, or DEFERRABLE + SET CONSTRAINTS DEFERRED.

COMBINE & SUMMARIZE

Joins · GROUP BY · HAVING

- Order: SELECT·FROM·**WHERE**(rows)·GROUP BY·**HAVING**(groups)·ORDER BY. WHERE can't use aggregates; move them to HAVING.
- INNER=matches only · LEFT=all left, right NULL if none · self-join=2 aliases (`e.manager_id=m.emp_id`).
- Every non-aggregate SELECT col must be in GROUP BY. Qualify shared names.
- LEFT no-match: `COUNT(*)`=1 (padded), `COUNT(right.col)`=0; HAVING on AVG/SUM drops empty LEFT groups (keep with COALESCE).
- `COUNT` w/o GROUP BY = 1 row (0 if none); with GROUP BY an **empty group never appears**.

Reference — PL/pgSQL / Views / Indexes / Transactions

functions · exceptions · types · views · keys · ACID · locking · security

PROCEDURAL CODE

PL/pgSQL

```
DO $b$ DECLARE _t INT:=0;
BEGIN FOR k IN 1..10 LOOP _t:=_t+k; END LOOP;
  IF _t>50 THEN RAISE INFO 'big %',_t; ELSIF _t=0 THEN ..;
  ELSE ..; END IF; END; $b$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION f(_p TYPE,_o TYPE=dflt)
RETURNS TYPE AS $fb$ DECLARE _v TYPE:=init;
BEGIN .. RETURN value; END; $fb$ LANGUAGE plpgsql;
```

- `:=` assigns; `;` ends each stmt; tags match. `ELSIF`, `END IF;`. **EXIT** leaves loop, **RETURN** ends fn. Every path returns.
- Defaults last; prefix vars `_`. `SELECT c INTO _v` = NULL if no row; MAX/MIN over 0 rows = NULL, COUNT = 0. NULL loop bound = error. DROP FUNCTION by arg types.

VALIDATE, DON'T CRASH

Exceptions & sentinels

- Check order: **arguments → existence → value**. Return **negative codes** (a real salary is positive); comment each code.
- A failed cast aborts the statement — wrap it in its own `BEGIN..EXCEPTION..END;`.
- `'Hi'::INT` → `invalid_text_representation`; past `NUMERIC(10,2)` max 99999999.99 → `numeric_value_out_of_range`.
- `IF NOT FOUND` after UPDATE/SELECT = no such row.

1-BASED · DATE PARTS

Arrays · dates · strings

fn	does
`a||b` · `LOWER/UPPER/LENGTH`	concat · case/len
`TRIM/LTRIM/RTRIM`	strip both/left/right
`POSITION('x' IN s)`	1-based, 0=not found
`LEFT/RIGHT(s,n)` · `SUBSTRING(s FROM 4 FOR 8)`	ends · slice
`ASCII/CHR`	'R'=82; '0'-'9'=48-57

- Arrays **1-based**; `CARDINALITY` =len (NULL→NULL); test `x = ANY(arr)`. `EXTRACT(MONTH/DAY/DOY FROM d)`. Fixed dates only.

STORED QUERY

Views

- `CREATE VIEW v (c1,c2) AS SELECT..; DROP VIEW v;` Rows computed on use; dropping leaves base tables. Uses: **data independence, security, convenience**.
- **Updatable** (SQL-92): 1 base table, select+project only, no aggregates. Insert → NULL in hidden cols (fails if PK/NOT NULL). **WITH CHECK OPTION** rejects rows leaving the condition.
- Views with **UNION/INTERSECT/EXCEPT** can't be inserted into. A view using a col blocks DROP COLUMN (RESTRICT).

CONSTRAINTS VS STRUCTURES

Keys & indexes

- **Superkey** ⊇ key · **candidate** minimal · **primary** chosen. Keys=constraints; index=performance (search key need not be unique).
- **Clustered**: data order=index order, ≤1/table, range=few contiguous pages. **Secondary/unclustered**: ~1 page/match (can lose to scan). **Unique** index contains a candidate key. **Index-only**: from entries alone.
- **Hash**: equality on **full** key only. **B+ tree**: equality+range on a **key prefix** — `<age,sal>` can't do `sal>40`. Cost = root→leaf path + leaves read; height ~3–4.
- Composite bigger, updated when any field changes. SQL std has no CREATE INDEX. Entry forms: record · `<key,rid>` · `<key,rid-list>`.

ACID · ISOLATION · LOCKS

Transactions & triggers

- **A**tomic · **C**onsistent · **I**solation (=some serial order) · **D**urable.

Level	Dirty	Unrep.	Phantom
READ UNCOMMITTED	maybe	maybe	maybe
READ COMMITTED	no	maybe	maybe
REPEATABLE READ	no	no	maybe
SERIALIZABLE	no	no	no

- Conflicts (same obj, ≥1 write): WR=dirty, RW=unrepeatable, WW=overwrite. Phantom=new matching row between reads.
- **Strict 2PL**: S to read, X to write, hold all to commit/abort; deadlock=cycle, DBMS aborts one. COMMIT/ROLLBACK/SAVEPOINT s/ROLLBACK TO s. A function call = one atomic unit; PREPARE holds 1 stmt.
- **Trigger**=Event+Condition+Action; before/after/instead of; per row/stmt. Proc params IN/OUT/INOUT.

SCAN THE NIGHT BEFORE

Common mistakes → fixes

Wrong	Right
`UPDATE t (col) SET`	`UPDATE t SET`
`SET e.salary=`	`SET salary=`
`WHERE (SELECT 1..)`	`WHERE EXISTS(..)`
EXISTS, no correlation	add `AND d.id=e.id`
`ALTER t ALTER col`	`ALTER TABLE t ALTER COLUMN`
`LIKE 'research'`	`ILIKE '%research%'`
`a+b` · `col=NULL`	`a||b` · `col IS NULL`
`ELSEIF` /no END IF · idx 0	`ELSIF` / `END IF;` · idx 1
aggregate in WHERE · RETURN in loop	HAVING · `EXIT`

Worked Problems — DML / DDL / Queries

official Exam 2 questions, solved

Q1 UPDATE + ILIKE + SUBQUERY

Double Research salaries

Employees of departments whose name contains "Research" (case-insensitive): double salary.

```
UPDATE Employees SET salary = salary * 2
WHERE dept_id IN (SELECT dept_id FROM Departments
                  WHERE dept_name ILIKE '%research%');
-- EXISTS form (needs the correlation line):
UPDATE Employees e SET salary = salary * 2
WHERE EXISTS (SELECT 1 FROM Departments d
   WHERE d.dept_id = e.dept_id AND d.dept_name ILIKE '%sales%');
```

▸ Inner query = matching dept ids; outer UPDATE hits their employees.

Q2 DELETE + NOT EXISTS

Drop empty zero-budget depts

Delete departments with budget 0 AND no employees.

```
DELETE FROM Departments d
WHERE d.budget = 0.00
  AND NOT EXISTS (SELECT 1 FROM Employees e
                  WHERE e.dept_id = d.dept_id);
-- scalar-count variant:
  AND (SELECT COUNT(*) FROM Employees e
       WHERE e.dept_id = d.dept_id) = 0;
```

▸ "No children" ⇒ NOT EXISTS (or COUNT(*) = 0).

Q3–Q5 ALTER TABLE

Three structure changes

Add NOT NULL to dept_name; drop manager_id; add salary_expectation.

```
ALTER TABLE Departments ALTER COLUMN dept_name SET NOT NULL;
ALTER TABLE Employees   DROP COLUMN manager_id;
ALTER TABLE Employees
  ADD COLUMN salary_expectation DECIMAL(15,2) NOT NULL DEFAULT 0.00;
```

▸ A NOT NULL column on a filled table needs a DEFAULT; fix NULLs before SET NOT NULL.

Q10 JOIN + GROUP BY + HAVING

Per-department stats

department_name, employees_no, max_salary — only depts whose average salary > 5000.

```
SELECT d.dept_name AS department_name,
       COUNT(e.emp_id) AS employees_no,
       MAX(e.salary)   AS max_salary
FROM Departments d
JOIN Employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 5000.00;
-- keep empty depts: LEFT JOIN + COALESCE(SUM(e.salary),0.00)
```

▸ Aggregate filter → HAVING. Group by PK + any selected label column.

extra CORRELATED SUBQUERY / HAVING

Two common query shapes

Above own-dept average; department payroll over budget.

```
SELECT e.emp_id, e.fname, e.lname, e.salary
FROM Employees e
WHERE e.salary > (SELECT AVG(e2.salary) FROM Employees e2
                  WHERE e2.dept_id = e.dept_id);

SELECT d.dept_name, d.budget, SUM(e.salary) AS payroll
FROM Departments d JOIN Employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name, d.budget
HAVING SUM(e.salary) > d.budget;
```

▸ Correlated subquery recomputes per employee's dept; group columns you also select.

Q8 VIEW + SELF-JOIN

Employee info view

View: id, full name, department, salary, manager's full name.

```
CREATE OR REPLACE VIEW Employee_Info AS
SELECT e.emp_id, e.fname||' '||e.lname AS full_name,
       d.dept_name, e.salary,
       COALESCE(m.fname||' '||m.lname,'N/A') AS manager_full_name
FROM Employees e
JOIN Departments d ON d.dept_id = e.dept_id
LEFT JOIN Employees m ON m.emp_id = e.manager_id;
```

▸ Self-join Employees m for the manager; LEFT JOIN keeps the top manager.

Worked Problems — Functions / Transactions / Security

validation · arrays · info string · atomic updates · privileges

Q6 VALIDATE + CAST + SENTINELS

f_SetSalary

Validate id & salary text, set salary; success returns salary, errors return negative codes.

```
CREATE OR REPLACE FUNCTION f_SetSalary(_emp_id BIGINT,
       _new_salary VARCHAR(20)) RETURNS DECIMAL(10,2) AS $fb$
DECLARE _salary DECIMAL(10,2) := NULL;
BEGIN
  IF _emp_id IS NULL OR NOT EXISTS
     (SELECT 1 FROM Employees WHERE emp_id=_emp_id)
  THEN RETURN -1.00; END IF;              -- bad id / none
  IF _new_salary IS NULL THEN RETURN -2.00; END IF;
  BEGIN                                   -- risky cast, own block
    _salary := TRIM(_new_salary)::DECIMAL(10,2);
  EXCEPTION
    WHEN invalid_text_representation THEN RETURN -2.00;
    WHEN numeric_value_out_of_range  THEN RETURN -3.00;
  END;
  IF _salary < 0 THEN RETURN -4.00; END IF;
  UPDATE Employees SET salary=_salary WHERE emp_id=_emp_id;
  RETURN _salary;
END; $fb$ LANGUAGE plpgsql;
```

▸ Order: argument → existence → value. Cast isolated so a bad string returns a code, not a crash.

Q11 ARRAY + EXTRACT + ANY

f_IsHoliday

TRUE if the date is a fixed holiday.

```
CREATE OR REPLACE FUNCTION f_IsHoliday(_date DATE)
RETURNS BOOLEAN AS $fb$
DECLARE _h INTEGER[] := ARRAY[101,704,1111,1225]; -- mo*100+day
        _k INTEGER;
BEGIN
  IF _date IS NULL THEN RETURN FALSE; END IF;
  _k := EXTRACT(MONTH FROM _date)*100 + EXTRACT(DAY FROM _date);
  RETURN _k = ANY(_h);              -- or FOR-loop compare
END; $fb$ LANGUAGE plpgsql;
```

▸ Encode each holiday as month*100+day; one `= ANY` membership test.

Quiz 2 EXISTENCE VS EMPTY BRANCHES

f_GetDeptInfo

Return a sentence: dept name, head, count, avg; handle NULL id / missing / empty.

```
IF p_dept_id IS NULL THEN RETURN 'ID cannot be NULL'; END IF;
SELECT COUNT(*) INTO v_exists FROM Departments
 WHERE dept_id=p_dept_id;
IF v_exists=0 THEN RETURN 'Department does not exist'; END IF;
SELECT dept_name,dept_head INTO v_name,v_head
 FROM Departments WHERE dept_id=p_dept_id;
SELECT COUNT(*), COALESCE(AVG(salary),0.00)
 INTO v_cnt,v_avg FROM Employees WHERE dept_id=p_dept_id;
IF v_cnt=0 THEN RETURN 'has 0 employees'; END IF;  -- empty
RETURN v_name||' has '||v_cnt||' employees; avg '||v_avg;
```

▸ Three separate branches: NULL arg vs. missing dept vs. empty dept. COALESCE guards head/avg.

Q12 AGGREGATE + THRESHOLD

f_getLargestSalariedEmployees

Count employees earning ≥ 95% of their department's max salary.

```
CREATE OR REPLACE FUNCTION f_getLargestSalariedEmployees(
       _dept_id SMALLINT) RETURNS SMALLINT AS $fb$
DECLARE _max DECIMAL(10,2); _n INTEGER;
BEGIN
  IF _dept_id IS NULL THEN RETURN -1; END IF;      -- bad arg
  SELECT MAX(salary) INTO _max FROM Employees
   WHERE dept_id=_dept_id;
  IF _max IS NULL THEN RETURN 0; END IF;           -- empty dept
  SELECT COUNT(emp_id) INTO _n FROM Employees
   WHERE dept_id=_dept_id AND salary >= 0.95*_max;
  RETURN _n;
END; $fb$ LANGUAGE plpgsql;
```

▸ MAX is NULL ⇒ empty department (sentinel 0); -1 for a bad argument.

Q9 TWO UPDATES, ALL-OR-NONE

Atomic transaction

Set Departments.dept_head and the head's dept_id together; abort on any failure.

```
DO $B$ DECLARE _ret INTEGER;
BEGIN
  UPDATE Departments SET dept_head=2317 WHERE dept_id=19
    RETURNING dept_id INTO _ret;
  IF _ret IS NULL THEN RAISE EXCEPTION 'Dept not found'; END IF;
  _ret := NULL;
  UPDATE Employees SET dept_id=19 WHERE emp_id=2317
    RETURNING emp_id INTO _ret;
  IF _ret IS NULL THEN RAISE EXCEPTION 'Emp not found'; END IF;
END; $B$ LANGUAGE plpgsql;
```

▸ A DO block is one unit — any RAISE EXCEPTION rolls back both. Plain SQL: BEGIN; .. ; COMMIT;

Q7 COLUMN SECURITY

Hide the salary column

Everyone reads non-salary columns; only approved users read salary.

```
REVOKE SELECT ON Employees FROM PUBLIC;
GRANT SELECT(emp_id,fname,lname,dept_id,manager_id)
      ON Employees TO PUBLIC;
GRANT SELECT(salary) ON Employees TO salary_readers;
-- or: a view without salary, grant the view to PUBLIC.
```

▸ Column-level GRANT exposes only the listed columns; put trusted users in a role.